# Table of Contents


| # | Program Title | Unit | Page |
|---|---|---|---|
| 1 | Hello Artificial Intelligence | Unit 1 | 3 |
| 2 | NumPy Array Operations | Unit 2 | 4 |
| 3 | Pandas DataFrame – Create & Analyse | Unit 2 | 5 |
| 4 | Data Visualisation – Bar & Pie Chart | Unit 2 | 6 |
| 5 | Data Cleaning with Pandas | Unit 2 | 7 |
| 6 | Linear Regression | Unit 3 | 8 |
| 7 | K-Nearest Neighbour (KNN) Classifier | Unit 3 | 9 |
| 8 | Decision Tree Classifier | Unit 3 | 10 |
| 9 | K-Means Clustering | Unit 3 | 11 |
| 10 | Confusion Matrix & Accuracy | Unit 3 | 12 |
| 11 | Image Read & Display using OpenCV | Unit 4 | 13 |
| 12 | Image Grayscale Conversion | Unit 4 | 14 |
| 13 | Edge Detection using Canny | Unit 4 | 15 |
| 14 | Face Detection using Haar Cascade | Unit 4 | 16 |
| 15 | Text Tokenisation using NLTK | Unit 5 | 17 |
| 16 | Stop Word Removal | Unit 5 | 18 |
| 17 | Stemming & Lemmatization | Unit 5 | 19 |
| 18 | Sentiment Analysis using TextBlob | Unit 5 | 20 |
| 19 | Word Cloud Generation | Unit 5 | 21 |
| 20 | Chatbot using if-else Logic | Unit 5 | 22 |
| – | Viva Questions & Answers | All | 23 |

# UNIT 1 — Introduction to Artificial Intelligence

**Program 1** | **Hello Artificial Intelligence** | **Unit 1**

### ■ Aim

To write a Python program to display a welcome message for AI and accept user input.

### ■ Python Code

```
# Program 1: Hello Artificial Intelligence
print("Hello, Artificial Intelligence!")
print("Welcome to CBSE Class 12 AI Lab Manual")
print("AI is transforming our world every day.")
print()
name = input("Enter your name: ")
print(f"Hello {name}! Let us learn AI together.")
print("Prepared by: AI Logic School")
```

### ■■ Expected Output

```
Hello, Artificial Intelligence!
Welcome to CBSE Class 12 AI Lab Manual
AI is transforming our world every day.

Enter your name: Priya
Hello Priya! Let us learn AI together.
Prepared by: AI Logic School
```

# UNIT 2 — Data Science & Pandas

## Program 2 NumPy Array Operations — Unit 2

### ■ Aim

To demonstrate basic NumPy array creation and arithmetic operations used in Data Science.

### ■ Python Code

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])

print("Array A:", a)
print("Array B:", b)
print("Sum      :", a + b)
print("Difference:", a - b)
print("Product  :", a * b)
print("Mean of A:", np.mean(a))
print("Max of A :", np.max(a))
print("Min of A :", np.min(a))
print("Shape    :", a.shape)
```

### ■■ Expected Output

```
Array A: [10 20 30 40 50]
Array B: [1 2 3 4 5]
Sum      : [11 22 33 44 55]
Difference: [ 9 18 27 36 45]
Product  : [ 10  40  90 160 250]
Mean of A: 30.0
Max of A : 50
Min of A : 10
Shape    : (5,)
```

---

## Program 3 Pandas DataFrame – Create & Analyse — Unit 2

### ■ Aim

To create a Pandas DataFrame of student records and perform basic data analysis operations.

### ■ Python Code

```
import pandas as pd

data = {
    'Name'  : ['Aarav','Priya','Rohan','Meena','Karan'],
    'Marks' : [85, 92, 78, 96, 88],
```

```
    'Grade' : ['A','A+','B+','A+','A']
}
df = pd.DataFrame(data)

print("Student Records:")
print(df)
print()
print("Average Marks:", df['Marks'].mean())
print("Highest Marks:", df['Marks'].max())
print()
print("Student with highest marks:")
print(df[df['Marks'] == df['Marks'].max()])
```

### ■■ Expected Output

```
Student Records:
    Name  Marks Grade
0  Aarav     85     A
1  Priya     92    A+
2  Rohan     78    B+
3  Meena     96    A+
4  Karan     88     A

Average Marks: 87.8
Highest Marks: 96

Student with highest marks:
    Name  Marks Grade
3  Meena     96    A+
```

---

## Program 4 — Data Visualisation – Bar & Pie Chart

Unit 2

### ■ Aim

To create bar chart and pie chart using Matplotlib to visualise student marks data.

### ■ Python Code

```
import matplotlib.pyplot as plt

names  = ['Aarav','Priya','Rohan','Meena','Karan']
marks  = [85, 92, 78, 96, 88]

# Bar Chart
plt.figure(figsize=(8,4))
plt.bar(names, marks, color=['blue','green','red','purple','orange'])
plt.title('Student Marks – Bar Chart')
plt.xlabel('Student Name')
plt.ylabel('Marks')
plt.ylim(0, 100)
plt.tight_layout()
plt.show()

# Pie Chart
plt.figure(figsize=(6,6))
plt.pie(marks, labels=names, autopct='%1.1f%%', startangle=90)
plt.title('Marks Distribution – Pie Chart')
plt.show()
```

### ■■ Expected Output

```
[Bar chart window opens showing coloured bars for each student]
[Pie chart window opens showing percentage distribution of marks]
Charts displayed successfully!
```

---

## Program 5 — Data Cleaning with Pandas

Unit 2

### ■ Aim

To demonstrate data cleaning techniques like handling missing values and removing duplicates using Pandas.

### ■ Python Code

```
import pandas as pd
import numpy as np

data = {
    'Name'  : ['Alice','Bob','Alice','Charlie',None],
    'Age'   : [25, None, 25, 30, 22],
    'Score' : [85, 90, 85, None, 78]
}
```

```
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)
print()
print("Missing values:")
print(df.isnull().sum())

df_clean = df.drop_duplicates()
df_clean = df_clean.fillna({'Age': df['Age'].mean(),
                            'Score': df['Score'].mean(),
                            'Name': 'Unknown'})
print()
print("Cleaned DataFrame:")
print(df_clean)
```

### ■■ Expected Output

```
Original DataFrame:
      Name   Age  Score
0    Alice  25.0   85.0
1      Bob   NaN   90.0
2    Alice  25.0   85.0
3  Charlie  30.0    NaN
4     None  22.0   78.0

Missing values:
Name     1
Age      1
Score    1

Cleaned DataFrame:
      Name   Age  Score
0    Alice  25.0   85.0
1      Bob   25.5   90.0
3  Charlie  30.0   84.5
4  Unknown  22.0   78.0
```

# UNIT 3 — Machine Learning

## Program 6 — Linear Regression

**Unit 3**

### ■ Aim

To implement a simple Linear Regression model to predict marks based on hours studied using scikit-learn.

### ■ Python Code

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

X = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1)
y = np.array([35,45,55,65,70,78,85,92])

model = LinearRegression()
model.fit(X, y)

print("Model trained successfully!")
print(f"Slope (m)    : {model.coef_[0]:.2f}")
print(f"Intercept (b): {model.intercept_:.2f}")

hours = np.array([[9]])
pred  = model.predict(hours)
print(f"Predicted marks for 9 hours: {pred[0]:.1f}")

plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', label='Regression Line')
plt.xlabel('Hours Studied'); plt.ylabel('Marks')
plt.title('Linear Regression: Hours vs Marks')
plt.legend(); plt.show()
```

### ■■ Expected Output

```
Model trained successfully!
Slope (m)    : 7.98
Intercept (b): 26.07
Predicted marks for 9 hours: 97.9
[Graph window opens with scatter plot and regression line]
```

---

## Program 7 — K-Nearest Neighbour (KNN) Classifier

**Unit 3**

### ■ Aim

To implement KNN classification algorithm using the Iris dataset from scikit-learn library.

### ■ Python Code

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred   = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("KNN Classifier on Iris Dataset")
print(f"Training samples : {len(X_train)}")
print(f"Testing  samples : {len(X_test)}")
print(f"Accuracy         : {accuracy*100:.2f}%")
print("Classes:", iris.target_names)
```

**■■ Expected Output**

```
KNN Classifier on Iris Dataset
Training samples : 105
Testing  samples : 45
Accuracy         : 97.78%
Classes: ['setosa' 'versicolor' 'virginica']
```

## Program 8 — Decision Tree Classifier — Unit 3

### ■ Aim

To build a Decision Tree classification model on the Iris dataset and visualise the tree.

### ■ Python Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train,X_test,y_train,y_test = train_test_split(
    X, y, test_size=0.3, random_state=0)

dt = DecisionTreeClassifier(max_depth=3, random_state=0)
dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)
print("Decision Tree Classifier")
print(f"Accuracy: {accuracy_score(y_test,y_pred)*100:.2f}%")
print()
print("Tree Structure:")
print(export_text(dt, feature_names=iris.feature_names))
```

### ■■ Expected Output

```
Decision Tree Classifier
Accuracy: 97.78%

Tree Structure:
|--- petal length (cm) <= 2.45
|   |--- class: 0  (setosa)
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1  (versicolor)
|   |--- petal width (cm) > 1.75
|   |   |--- class: 2  (virginica)
```

---

## Program 9 — K-Means Clustering — Unit 3

### ■ Aim

To implement K-Means clustering algorithm to group data points into clusters using scikit-learn.

### ■ Python Code

```
import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

np.random.seed(42)
X = np.vstack([
    np.random.randn(50,2) + [0,0],
    np.random.randn(50,2) + [5,5],
    np.random.randn(50,2) + [0,5]
])

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)
labels   = kmeans.labels_
centers  = kmeans.cluster_centers_

print("K-Means Clustering")
print(f"Number of clusters: 3")
print(f"Cluster centers:")
for i, c in enumerate(centers):
    print(f"  Cluster {i}: ({c[0]:.2f}, {c[1]:.2f})")

plt.scatter(X[:,0], X[:,1], c=labels, cmap='Set1')
plt.scatter(centers[:,0], centers[:,1],
            marker='X', s=200, c='black', label='Centroids')
plt.title('K-Means Clustering')
plt.legend(); plt.show()
```

**■■ Expected Output**

```
K-Means Clustering
Number of clusters: 3
Cluster centers:
  Cluster 0: (0.08, 0.03)
  Cluster 1: (4.95, 5.00)
  Cluster 2: (0.05, 4.97)
[Scatter plot with 3 coloured clusters displayed]
```

**Program 10** | **Confusion Matrix & Accuracy Score** | **Unit 3**

## ■ Aim

To evaluate a machine learning classification model using confusion matrix and accuracy score.

## ■ Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion_matrix,
                             accuracy_score,
                             classification_report)

iris = load_iris()
X_train,X_test,y_train,y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print()
print(f"Accuracy: {accuracy_score(y_test,y_pred)*100:.2f}%")
print()
print("Classification Report:")
print(classification_report(y_test, y_pred,
      target_names=iris.target_names))
```

## ■■ Expected Output

```
Confusion Matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

Accuracy: 97.78%

Classification Report:
              precision  recall  f1-score  support
    setosa       1.00    1.00    1.00       19
versicolor       1.00    0.92    0.96       13
 virginica       0.93    1.00    0.96       13
  accuracy                       0.98       45
```

# UNIT 4 — Computer Vision (OpenCV)

## Program 11 — Image Read & Display using OpenCV

Unit 4

### ■ Aim

To read an image file and display it using OpenCV library in Python.

### ■ Python Code

```
import cv2

# Read image
img = cv2.imread('sample.jpg')

# Display image info
print("Image Shape:", img.shape)
print("Height     :", img.shape[0])
print("Width      :", img.shape[1])
print("Channels   :", img.shape[2])

# Display image
cv2.imshow('Original Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
print("Image displayed successfully!")
```

### ■■ Expected Output

```
Image Shape: (480, 640, 3)
Height     : 480
Width      : 640
Channels   : 3
[Image window opens showing the original image]
Image displayed successfully!
```

---

## Program 12 — Image Grayscale Conversion

Unit 4

### ■ Aim

To convert a colour image to grayscale and display both images using OpenCV.

### ■ Python Code

```
import cv2

img      = cv2.imread('sample.jpg')
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("Original  shape:", img.shape)
print("Grayscale shape:", gray_img.shape)

cv2.imshow('Original Image',  img)
```

```
cv2.imshow('Grayscale Image', gray_img)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imwrite('gray_output.jpg', gray_img)
print("Grayscale image saved as gray_output.jpg")
```

**■■ Expected Output**

```
Original  shape: (480, 640, 3)
Grayscale shape: (480, 640)
[Two windows: Original colour image & Grayscale image]
Grayscale image saved as gray_output.jpg
```

---

## Program 13 — Edge Detection using Canny

**Unit 4**

### ■ Aim

To detect edges in an image using the Canny edge detection algorithm in OpenCV.

### ■ Python Code

```
import cv2

img  = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply Canny edge detection
edges = cv2.Canny(gray, threshold1=100, threshold2=200)

print("Edge detection applied!")
print("Edge image shape:", edges.shape)

cv2.imshow('Original',    gray)
cv2.imshow('Edges (Canny)', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imwrite('edges_output.jpg', edges)
print("Edge image saved!")
```

### ■■ Expected Output

```
Edge detection applied!
Edge image shape: (480, 640)
[Two windows: Grayscale original & Edge detected image]
Edge image saved!
```

---

## Program 14 — Face Detection using Haar Cascade

**Unit 4**

### ■ Aim

To detect human faces in an image using OpenCV's pre-trained Haar Cascade classifier.

### ■ Python Code

```
import cv2

# Load classifier
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

img  = cv2.imread('people.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(
    gray, scaleFactor=1.1, minNeighbors=5, minSize=(30,30))
```

```
print(f"Number of faces detected: {len(faces)}")

# Draw rectangles
for (x,y,w,h) in faces:
    cv2.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)

cv2.imshow('Face Detection', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

### ■■ Expected Output

```
Number of faces detected: 3
[Image window opens with green rectangles drawn around detected faces]
```

---

# UNIT 5 — Natural Language Processing (NLP)

## Program 15 — Text Tokenisation using NLTK

Unit 5

### ■ Aim

To perform word and sentence tokenisation on a given text using the NLTK library.

### ■ Python Code

```
import nltk
nltk.download('punkt', quiet=True)
from nltk.tokenize import word_tokenize, sent_tokenize

text = ("Artificial Intelligence is the future. "
        "Python is the best language for AI. "
        "CBSE has introduced AI as a subject.")

print("Original Text:")
print(text)
print()
sentences = sent_tokenize(text)
print("Sentence Tokens:")
for i, s in enumerate(sentences, 1):
    print(f"  {i}. {s}")
print()
words = word_tokenize(text)
print("Word Tokens:", words)
print("Total words:", len(words))
```

### ■■ Expected Output

```
Original Text:
Artificial Intelligence is the future. Python is the best
language for AI. CBSE has introduced AI as a subject.

Sentence Tokens:
  1. Artificial Intelligence is the future.
  2. Python is the best language for AI.
  3. CBSE has introduced AI as a subject.

Word Tokens: ['Artificial','Intelligence','is','the','future',...]
Total words: 22
```

---

## Program 16 — Stop Word Removal

Unit 5

### ■ Aim

To remove common stop words from a given sentence using NLTK.

### ■ Python Code

```
import nltk
nltk.download('stopwords', quiet=True)
nltk.download('punkt',     quiet=True)
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text  = "Artificial Intelligence is the future of the world"
words = word_tokenize(text.lower())
stop  = set(stopwords.words('english'))

filtered = [w for w in words if w not in stop]

print("Original  :", words)
print("Stop words:", list(stop)[:8], "...")
print("Filtered  :", filtered)
print()
print("Words removed:", len(words) - len(filtered))
```

### ■■ Expected Output

```
Original  : ['artificial','intelligence','is','the','future','of','the','world']
Stop words: ['i','me','my','myself','we','our','ours','ourselves'] ...
Filtered  : ['artificial','intelligence','future','world']

Words removed: 4
```

## Program 17 — Stemming & Lemmatization — Unit 5

### ■ Aim

To apply stemming and lemmatization on words using NLTK to find root forms.

### ■ Python Code

```
import nltk
nltk.download('wordnet', quiet=True)
from nltk.stem import PorterStemmer, WordNetLemmatizer

words = ['running','runs','ran','easily','fairly',
         'studies','studying','studied','better','caring']

stemmer    = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print(f"{'Word':<12} {'Stemmed':<14} {'Lemmatized'}")
print("-" * 40)
for w in words:
    stemmed    = stemmer.stem(w)
    lemmatized = lemmatizer.lemmatize(w, pos='v')
    print(f"{w:<12} {stemmed:<14} {lemmatized}")
```

### ■■ Expected Output

```
Word         Stemmed        Lemmatized
----------------------------------------
running      run            run
runs         run            run
ran          ran            run
easily       easili         easily
fairly       fairli         fairly
studies      studi          study
studying     studi          study
studied      studi          study
better       better         better
caring       care           care
```

---

## Program 18 — Sentiment Analysis using TextBlob — Unit 5

### ■ Aim

To perform sentiment analysis on text data using the TextBlob library in Python.

### ■ Python Code

```
from textblob import TextBlob

sentences = [
    "I love Artificial Intelligence. It is amazing!",
```

```
    "This movie was terrible and very boring.",
    "The weather today is okay.",
    "Python programming is fun and easy to learn.",
    "I hate waiting in long queues."
]

print(f"{'Text':<45} {'Polarity':>9} {'Sentiment'}")
print("-" * 65)
for s in sentences:
    blob      = TextBlob(s)
    polarity  = blob.sentiment.polarity
    sentiment = ("Positive" if polarity > 0
                 else "Negative" if polarity < 0
                 else "Neutral")
    print(f"{s[:44]:<45} {polarity:>9.2f}  {sentiment}")
```

### ■■ Expected Output

```
Text                                          Polarity  Sentiment
-----------------------------------------------------------------
I love Artificial Intelligence. It is amaz...     0.62  Positive
This movie was terrible and very boring.          -0.60  Negative
The weather today is okay.                         0.20  Positive
Python programming is fun and easy to learn.       0.50  Positive
I hate waiting in long queues.                    -0.50  Negative
```

## Program 19 — Word Cloud Generation — Unit 5

### ■ Aim

To generate a word cloud from a given text to visualise the most frequent words.

### ■ Python Code

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text = ('Artificial Intelligence Machine Learning Deep Learning '
        'Python Data Science Neural Network CBSE Class Twelve '
        'Algorithm Model Training Testing Accuracy Prediction '
        'NLP Computer Vision Robotics India Education Technology')

wc = WordCloud(width=800, height=400,
               background_color='white',
               colormap='plasma',
               max_words=50).generate(text)

plt.figure(figsize=(10,5))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud - AI Topics', fontsize=16)
plt.tight_layout()
plt.show()
print('Word Cloud generated successfully!')
```

### ■■ Expected Output

```
[Word Cloud image window opens showing AI-related words in different sizes]
Word Cloud generated successfully!
```

---

## Program 20 — Simple Chatbot using if-else Logic — Unit 5

### ■ Aim

To create a simple rule-based chatbot using if-else conditions in Python.

### ■ Python Code

```
def chatbot(user_input):
    user_input = user_input.lower()
    if "hello" in user_input or "hi" in user_input:
        return "Hello! I am AI Bot. How can I help you?"
    elif "name" in user_input:
        return "My name is AI Bot, your study assistant!"
    elif "ai" in user_input:
        return "AI stands for Artificial Intelligence."
    elif "python" in user_input:
```

```
        return "Python is the best language for AI & ML!"
    elif "cbse" in user_input:
        return "CBSE has AI as a subject from Class 8 to 12."
    elif "bye" in user_input or "exit" in user_input:
        return "Goodbye! Happy Learning!"
    else:
        return "Sorry, I did not understand. Ask me about AI!"
print("AI Chatbot Started! Type 'bye' to exit.")
print("-" * 40)
while True:
    user = input("You: ")
    resp = chatbot(user)
    print("Bot:", resp)
    if "bye" in user.lower() or "exit" in user.lower():
        break
```

### ■■ Expected Output

```
AI Chatbot Started! Type 'bye' to exit.
----------------------------------------
You: hello
Bot: Hello! I am AI Bot. How can I help you?
You: what is ai
Bot: AI stands for Artificial Intelligence.
You: tell me about python
Bot: Python is the best language for AI & ML!
You: bye
Bot: Goodbye! Happy Learning!
```

# VIVA QUESTIONS & ANSWERS

**Q1. What is Artificial Intelligence?**

Artificial Intelligence (AI) is the simulation of human intelligence in machines that are programmed to think, learn, and solve problems like humans.

**Q2. What is the difference between Supervised and Unsupervised Learning?**

Supervised Learning uses labelled data (e.g., Linear Regression, KNN). Unsupervised Learning finds hidden patterns in unlabelled data (e.g., K-Means Clustering).

**Q3. What is a DataFrame in Pandas?**

A DataFrame is a 2D labelled data structure in Pandas with rows and columns, similar to a spreadsheet or SQL table.

**Q4. What does train_test_split() do?**

It splits the dataset into training and testing sets. The model learns from training data and is evaluated on testing data.

**Q5. What is accuracy score in machine learning?**

Accuracy = (Correct Predictions / Total Predictions) x 100. It measures how well the model predicts on test data.

**Q6. What is Linear Regression?**

Linear Regression is a supervised ML algorithm that finds the best-fit line (y = mx + b) to predict continuous output values.

**Q7. What is KNN algorithm?**

K-Nearest Neighbours classifies a data point based on the majority class among its K nearest neighbours in the feature space.

**Q8. What is a Confusion Matrix?**

A confusion matrix is a table showing True Positives, True Negatives, False Positives, and False Negatives for a classification model.

**Q9. What is tokenisation in NLP?**

Tokenisation is the process of splitting text into smaller units called tokens — either words (word tokenisation) or sentences (sentence tokenisation).

**Q10. What are stop words?**

Stop words are common words (like 'is', 'the', 'a', 'in') that carry little meaning and are removed during NLP preprocessing.

**Q11. What is the difference between Stemming and Lemmatization?**

Stemming cuts words to their root form (running -> run, studies -> studi). Lemmatization returns the correct dictionary form (studies -> study).

**Q12. What is Sentiment Analysis?**

Sentiment Analysis determines the emotional tone of text — whether it is positive, negative, or neutral. It is used in product reviews and social media analysis.

**Q13. What is OpenCV?**

OpenCV (Open Source Computer Vision Library) is a Python library used for image processing, face detection, object detection, and video analysis.

### Q14. What is edge detection?

Edge detection identifies boundaries within images where there is a sudden change in pixel intensity. The Canny algorithm is commonly used.

### Q15. What is the difference between AI, ML, and Deep Learning?

AI is the broad field of making machines intelligent. ML is a subset where machines learn from data. Deep Learning is a subset of ML using neural networks with many layers.